



LiteLLM AI Gateway for Agentic AI

Features, configuration steps, tips and tricks for Technical AI Architects

Client Name: Philips

Date: 18 September 2026

Why do we need an AI gateway?

Agentic systems call many models, many times. Without a central control point, things become messy very fast.



Without a gateway

- Every agent keeps its own provider API keys
- Retry and fallback logic is copied in every service
- No single view of cost per team, agent or customer
- Safety checks (PII, prompt injection) done differently everywhere
- Changing a model means changing code



With LiteLLM gateway

- One endpoint, one virtual key per agent
- Load balancing, retries and fallbacks handled centrally
- Spend tracked per key, team, user and tag
- Guardrails applied the same way for all traffic
- Swap models by changing config, not code



When NOT to use it: single model from a single provider with no plans to change, or ultra-low-latency paths such as real-time voice under 150 ms. A direct call is simpler there.

What is LiteLLM?

Open-source gateway that puts all your LLMs, MCP tools and agents behind one OpenAI-compatible API



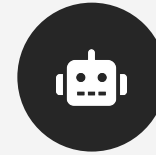
LLM Gateway

Call 140+ providers (Anthropic, OpenAI, Bedrock, Vertex, vLLM, Ollama) in one OpenAI format. Anthropic /v1/messages format also supported.



MCP Gateway

Central MCP endpoint for tools. Control which key or team can see which MCP server and tool.



A2A Agent Gateway

Register A2A agents (LangGraph, Bedrock AgentCore, Vertex, Pydantic AI). Same logging, guardrails and limits as LLM calls.

140+

providers supported

~8 ms

P95 overhead at 1k RPS

1 key

for your whole AI stack

Self-host

cloud, on-prem or air-gapped

Two ways to use it: Python SDK (library inside your code) or AI Gateway / Proxy (central service for all teams). This deck focuses on the Gateway.

Feature map at a glance

Nine building blocks you will configure as an architect



Unified API

Any model in OpenAI format; Claude traffic can use Anthropic format



Routing & fallbacks

Spread load across deployments, switch on errors or long context



Auto Router

Picks a model tier by prompt complexity, rules or keywords



Virtual keys & access

Keys per team, user, agent; restrict which models each can call



Budgets & spend

Cost per key, team, user, tag; daily and monthly caps



Rate limits

TPM, RPM, parallel requests, per-model and per-session limits



Guardrails

PII masking, prompt injection, moderation, tool permissions



MCP & A2A gateway

Central tools and agents with per-key access control



Observability

Admin UI logs, Langfuse, OpenTelemetry, Datadog, MLflow

Reference architecture for agentic AI

Agents talk only to the gateway. The gateway talks to models, tools and other agents.

YOUR AGENTS

LangGraph agents

Claude Agent SDK

n8n / apps / APIs

LiteLLM AI Gateway

1. Authenticate virtual key
2. Pre-call guardrails
3. Budget & rate-limit check
4. Router: pick deployment
5. Post-call guardrails
6. Log spend & trace

BACKENDS

● **Proprietary models**
Anthropic, OpenAI, Bedrock, Vertex

● **Open-source models**
vLLM, Ollama, TGI (self-hosted)

● **MCP servers**
Tools, data, actions

● **A2A agents**
Remote specialist agents



PostgreSQL
keys, teams, spend (mandatory)



Redis
counters & cache (multi-replica)



Langfuse / OTEL
traces across agent runs

Step 0 — Set up the foundation

Do these four things before you add any model or user

1

Use the official Docker image, pinned

Use a -stable tag or exact version. Never an unpinned pip install in production.

2

Connect PostgreSQL

Mandatory for virtual keys, teams, users and budgets.

3

Add Redis for more than one replica

Keeps spend and rate-limit counters in sync across pods.

4

Keep secrets in the gateway only

Master key and provider keys from a secret manager. Agents get only virtual keys.

```
# config.yaml
general_settings:
  master_key: os.environ/LITELLM_MASTER_KEY
  database_url: os.environ/DATABASE_URL

litellm_settings:
  cache: true
  cache_params:
    type: redis
    host: os.environ/REDIS_HOST
    port: 6379

# run (pinned image, not :latest)
docker run -p 4000:4000 \
  -v $(pwd)/config.yaml:/app/config.yaml \
  <litellm-image>:<pinned-stable-tag> \
  --config /app/config.yaml
```



Important: without a database, budgets fail open — requests keep going even after the limit is crossed. Only a startup warning is logged.

Step 1 — Register your models

Each entry in `model_list` = one deployment. `model_name` is the alias your agents will call.

```
model_list:
  # Proprietary
  - model_name: planner
    litellm_params:
      model: anthropic/claude-opus-5
      api_key: os.environ/ANTHROPIC_API_KEY
    model_info: { access_groups: ["frontier"] }

  # Open source, 2 replicas = auto load-balanced
  - model_name: executor-oss
    litellm_params:
      model: hosted_vllm/<your-oss-model>
      api_base: http://vllm-1:8000/v1
      input_cost_per_token: 0
      output_cost_per_token: 0
    model_info: { access_groups: ["oss"] }
  - model_name: executor-oss
    litellm_params:
      model: hosted_vllm/<your-oss-model>
      api_base: http://vllm-2:8000/v1
      input_cost_per_token: 0
      output_cost_per_token: 0
    model_info: { access_groups: ["oss"] }
```

Name by role, not vendor. Agents call planner, executor, judge. Swapping Claude for Qwen becomes a config change.

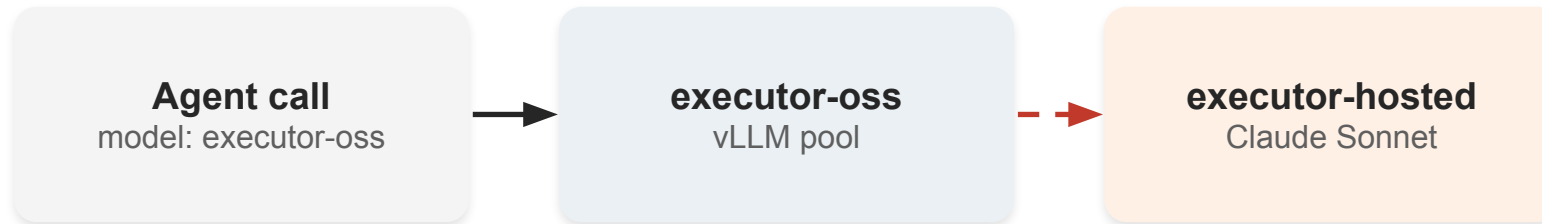
Same `model_name` twice = a load-balanced pool. Add replicas without touching agent code.

Set cost 0 on self-hosted models. Budget checks are skipped for zero-cost models, so a key that has exhausted its budget can still fall back to your own GPUs.

Use `access_groups` to grant a bundle of models to a team or key in one shot.

Routing, fallbacks and Auto Router

Let the gateway handle which deployment serves a call and what happens when it fails



on error / 429 / context too long

```
router_settings:
  routing_strategy: simple-shuffle # least-busy, latency-based-routing ...
  num_retries: 2
  fallbacks: [{ "executor-oss": ["executor-hosted"] }]
  context_window_fallbacks: [{ "executor-oss": ["executor-hosted"] }]
  enable_tag_filtering: true # e.g. tag "sensitive" -> on-prem
```



Auto Router

One alias that picks a model tier using complexity scoring, an LLM classifier, keyword rules or adaptive sampling. It reads the last few turns before choosing. Best for one-shot calls like classification, extraction, summaries.



Golden rule for agents: do not switch models in the middle of an agent session. Prompt cache, thinking blocks and tool-call history do not transfer. Fix one model per agent run; fall back only within the same capability tier, and test that the fallback model emits valid tool calls.

Step 2 — Users, teams and virtual keys

Access flows top-down. Give every agent its own key.

Organization

Enterprise only

Team

a product or workload

User

a developer or admin

Virtual key

one per agent / service

```
# 1. Team with models + team limits
POST /team/new
{ "team_alias": "agents-prod", "models": ["frontier","oss"],
  "max_budget": 500, "budget_duration": "30d",
  "tpm_limit": 2000000, "rpm_limit": 1000 }

# 2. Add a member (optional cap inside team)
POST /team/member_add
{ "team_id": "<id>", "member": {"role":"admin","user_id":"arun"},
  "max_budget_in_team": 100 }

# 3. One key per agent, only the models it needs
POST /key/generate
{ "key_alias": "research-agent", "team_id": "<id>",
  "models": ["planner","executor-oss"], "tags": ["research-agent"] }
```



Watch out: a key inside a team follows only team and team-member budgets. The owner's personal budget is ignored.



Test tip: rate limits do not apply to proxy admins. Test with an `internal_user` key, and never give the master key to an agent.

Step 3 — Rate limits and budgets

Set limits at every level. The tightest applicable limit wins.

Level	What you can set	Use it for
Global proxy	max_budget, budget_duration	Company-wide safety cap
Team	Budget, TPM, RPM, max parallel, per-model TPM/RPM	Each product or workload
Team member	max_budget_in_team	Developer cap inside a team
Virtual key	Budget windows, TPM/RPM, per-model TPM/RPM	Each agent or service
Customer	budget_id with budget, TPM/RPM	SaaS end users (no key needed)
Agent (A2A)	Agent TPM/RPM, session TPM/RPM, iterations, \$ per session	Runaway-loop protection

Stacked budget windows on one key

```
POST /key/update
{ "key": "sk-...",
  "budget_limits": [
    {"budget_duration": "24h",
     "max_budget": 10},
    {"budget_duration": "30d",
     "max_budget": 100} ],
  "model_rpm_limit":
    {"planner": 30} }
```



Why two windows? \$10/day stops a runaway loop on the same day. \$100/month stays the ceiling. One window alone cannot do both.

Controls that stop runaway agents

Agent loops can burn money quietly. These settings keep cost and throughput predictable.



Session caps (A2A)

max_iterations and max_budget_per_session cap each agent run. session_tpm_limit / session_rpm_limit cap its speed.



Budget reservation

On by default. Gateway estimates the maximum cost first and rejects the call if it would cross the budget. Protects against many parallel calls.



Estimated output tokens

If requests have no max_tokens, set default_estimated_output_tokens_per_model near your p95 output. Too low or too high both hurt.



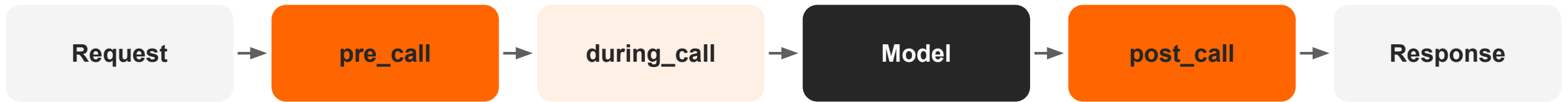
Fail-closed budgets

fail_closed_budget_enforcement: true checks spend against the DB when Redis is unhealthy. If spend cannot be verified, the call gets a 503.

```
POST /v1/agents { "agent_name": "offer-letter-agent", "agent_card_params": {...},  
                  "tpm_limit": 500000, "rpm_limit": 200, "session_tpm_limit": 50000, "session_rpm_limit": 50 }
```

Step 4 — Guardrails: how they work

A guardrail is a check that runs at a chosen point in the request lifecycle



pre_call: checks input before the model. **during_call:** runs in parallel; response held till it passes. **post_call:** checks input + output. **logging_only:** masks only in logs.

Three ways to attach a guardrail

default_on: true (OSS) — runs on every request; clients cannot switch it off. Your main enforcement tool.

guardrails param in request (OSS) — client chooses. Good for testing, not for enforcement.

Per key, per model, per team, Policies (Enterprise) — server-side scoping, lock teams from disabling.

```
guardrails:
- guardrail_name: pii-mask
  litellm_params:
    guardrail: presidio
    mode: pre_call
    default_on: true
    pii_entities_config:
      EMAIL_ADDRESS: MASK
      PHONE_NUMBER: MASK
      PERSON: MASK
```



Test cheaply: send mock_response in the body to run guardrails without calling the model; check header x-litellm-applied-guardrails.

Guardrail options — pick what fits

Start with built-in ones; add vendors when you need stronger detection



Built-in

- Content filter
- Prompt injection detection
- Sensitive Data Routing
- Tool Permission
- LLM-as-a-Judge
- Custom code / Python



PII / PHI

- Presidio masking
- MASK or BLOCK per entity
- Confidence thresholds per entity
- Secret redaction (Enterprise)



Cloud providers

- Bedrock Guardrails
- Azure Content Safety
- Google Model Armor
- OpenAI Moderation
- Microsoft Purview DLP



Security vendors

- Lakera, Aporia
- Pangea, Prisma AIRS
- Zscaler, CrowdStrike
- HiddenLayer, Lasso
- PromptGuard (self-host)



Bring your own

- Generic Guardrail API (any HTTP service)
- Teams can register, admin approves



Star pick: Sensitive Data Routing re-routes sensitive requests to an on-prem model instead of blocking them.



Check endpoints: e.g. Lakera v2 covers only /chat/completions, not /v1/messages, MCP or A2A.

Tips, tricks and common mistakes

A quick checklist before you go to production



Do this

- Role-based aliases (planner, executor, judge)
- One virtual key per agent, with tags
- 24h + 30d budget windows on every key
- default_on guardrails for PII and injection
- Keep sensitive data on self-hosted models
- Pass session / trace IDs to Langfuse or OTel
- Use Anthropic /v1/messages for Claude traffic
- Load test with non-admin keys



Avoid this

- Master key inside any agent or app
- Switching models mid-session
- DB-less deployment when you need budgets
- Unpinned pip install litellm (March 2026 PyPI compromise: 1.82.7, 1.82.8)
- Trusting client-side guardrail param for enforcement
- Using header tags (User-Agent) as a security boundary
- Gateway in sub-150 ms voice paths

Thank You

unext
